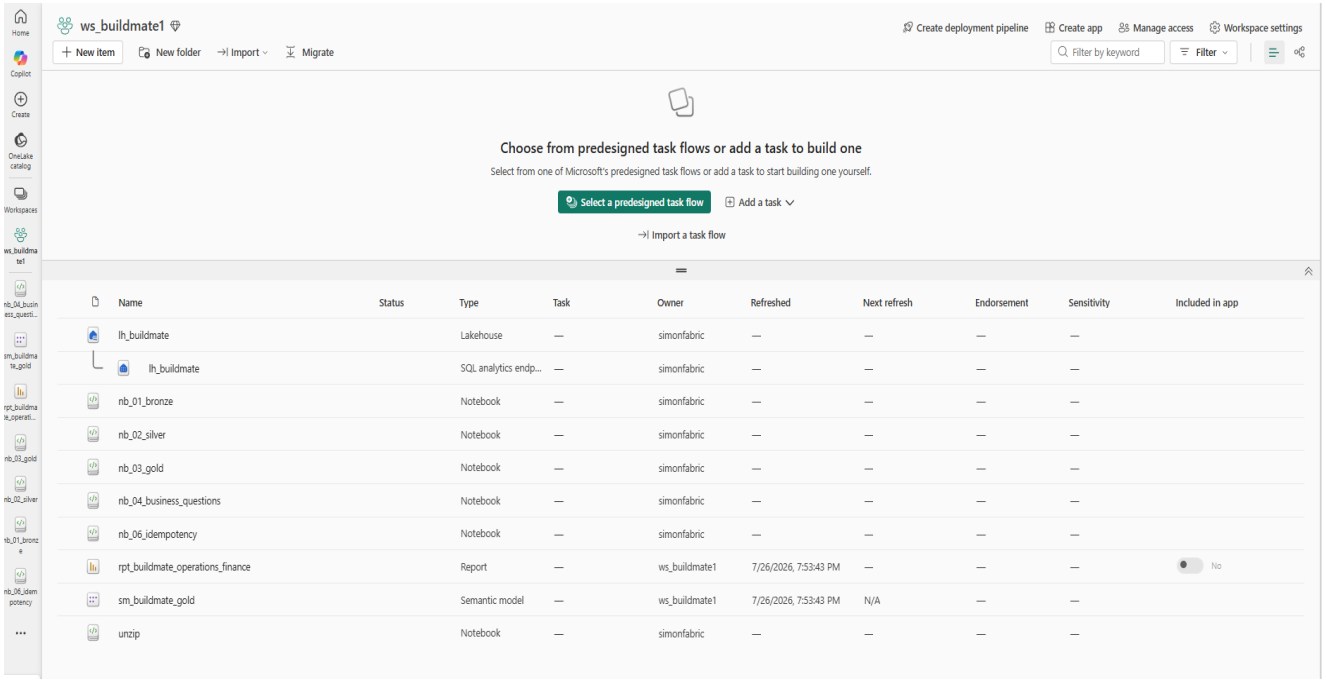
BuildMate Rentals

Microsoft Fabric End-to-End Submission Write-Up

Author	Simon Montenegro
Workspace	ws_buildmate1
Lakehouse	lh_buildmate
Architecture	OneLake Lakehouse with Bronze, Silver and Gold Delta tables

This document summarizes the full BuildMate Rentals implementation, the validation results, the issues resolved during development, and the main Microsoft Fabric concepts applied.



Executive Summary

The solution was built in Microsoft Fabric as an end-to-end medallion pipeline using one Lakehouse in OneLake. All raw source data was preserved in Bronze, cleaned and conformed in Silver, and transformed into a Gold star schema for operational and financial reporting. The most important engineering control was null-safe filtering, which preserved 175 active rentals that had no check-in timestamp. The Gold model reconciled approximately INR 4.98 crore of matched revenue and reported 175 machines currently out across the six depots.

Layer	Main purpose	Key result
Bronze	Faithful landing with lineage	Four raw tables; 31 rental files loaded by wildcard
Silver	Clean, standardise, deduplicate and quarantine	600 customers; 942 valid rentals; 3 quarantined
Gold	Conformed dimensions and honest facts	Five Gold tables; Delta OPTIMIZE and ZORDER applied

Task 1 - Bronze: Land Every Source Exactly as Received

The raw archive was expanded under the Files area of lh_buildmate. Every source column was read with an explicit string schema, and only two lineage fields were added: ingested_at and source_file. No trimming, casting, standardisation, filtering or deduplication was performed.

- bronze_customers: 602 rows
- bronze_depots: 6 rows
- bronze_rentals: 976 rows
- bronze_billing: 787 rows
- All 31 rental CSV files were loaded in one wildcard read using rentals_*.csv.

Re-sent day evidence

The source_file lineage proved that rentals_2026-06-10.csv and rentals_2026-06-10_resend.csv both landed in Bronze and contained the same rental IDs and row count. An initial exact-filename comparison returned an empty result because Fabric stored the full file URI. The validation was corrected by filtering source_file with contains('rentals_2026-06-10') and then extracting the filename for display.

Task 2 - Silver: Clean, Conform and Preserve Live Rentals

Silver first showed the damage in the raw data and then corrected it deliberately. Customers were deduplicated by CUSTOMER_ID and rentals by rental_id, keeping the latest lineage row. Customer and rental categories were standardised to their required conformed values.

Validation	Expected/actual result	Meaning
Customer deduplication	602 -> 600	Two excess customer registration rows removed
Rental deduplication	976 -> 945	The 31-row re-sent file was removed
Date parsing	273 would fail with one format; 0 after coalesce	Both dd-MM-yyyy and yyyy/MM/dd were parsed
Rental quality rule	942 valid; 3 quarantined	Impossible check-in-before-checkout rows isolated

Null safety	175 active rentals preserved	Blank check-in means still out, not invalid
Billing matching	779 matched; 8 unmatched	Left join preserved orphan bills in a side table

Amount parsing correction

The first amount-cleaning expression produced 474 parse failures because it retained the period in the 'Rs.' prefix. For example, 'Rs. 42,180.00' became '.42180.00', which contains two decimal points. The logic was corrected by first removing the Rs. prefix and then removing commas before casting to decimal(18,2). The final amount parse-failure assertion passed at zero.

Null-safe filtering

The final quality filter was `filter(~coalesce(bad, False))`. The bad expression is true only when checkin_ts is earlier than checkout_ts. When checkin_ts is null, coalesce replaces the unknown Boolean with false, so the row is treated as not bad and remains in Silver. The careless filter filter(~bad) would have silently removed all 175 currently-out rentals.

> *Key lesson: Spark filters retain only rows that evaluate to TRUE. A NULL Boolean is not TRUE, so null handling must be explicit.*

Task 3 - Gold: Build the Conformed Star and Tune Delta

Gold created business-ready objects that did not exist in any source. The dimensions and facts share conformed customer, depot and date keys so rental operations and billing can reconcile in one analytical model.

Gold table	Rows	Purpose
gold_dim_customer	600	Customer attributes plus tenure_years and tenure_band
gold_dim_depot	6	Clean depot lookup promoted as a dimension
gold_dim_date	30	Generated June 2026 calendar
gold_fact_rental	942	Returned and active rentals with duration and windowed asset days
gold_fact_billing	779	Parsed matched bills

- rental_duration_days is NULL for exactly the 175 still-out rentals.
- is_returned is 0 for those same 175 rows.
- asset_days_in_window is clipped to the 30-day reporting window and never exceeds 30.
- OPTIMIZE ... ZORDER was run on both fact tables and verified through DESCRIBE HISTORY.

> *The facts were not partitioned because they are very small (942 and 779 rows); partitioning would add metadata and small-file overhead. Duration remains NULL for active rentals because their final duration is unknown, while zero would be misleading.*

Before and after OPTIMIZE

Delta history was used to compare the version immediately before OPTIMIZE with the version created by OPTIMIZE. The row count remained unchanged, while operation metrics showed files removed, files added and bytes rewritten. Because the facts contain fewer than one thousand rows, file-layout evidence is more meaningful than elapsed-time benchmarking.

Task 4 - Business Payoff from the Gold Star

Six Spark SQL result sets were produced: average duration by asset type, utilisation by depot, revenue by payer type, revenue by depot, rental-type mix and machines currently out. The cross-system revenue-by-depot query reconciled exactly to gold_fact_billing, with total matched revenue of INR 49,775,401 (about 4.98 crore).

Revenue by depot

Depot	Bills	Revenue (INR)	Revenue per rental (INR)
Hadapsar Depot	216	10,785,755	49,934
Chakan Depot	152	10,587,483	69,654
Wakad Depot	143	9,605,530	67,172
Talegaon Depot	103	9,364,785	90,920
Kharadi Depot	96	5,621,595	58,558
Baramati Depot	69	3,810,253	55,221

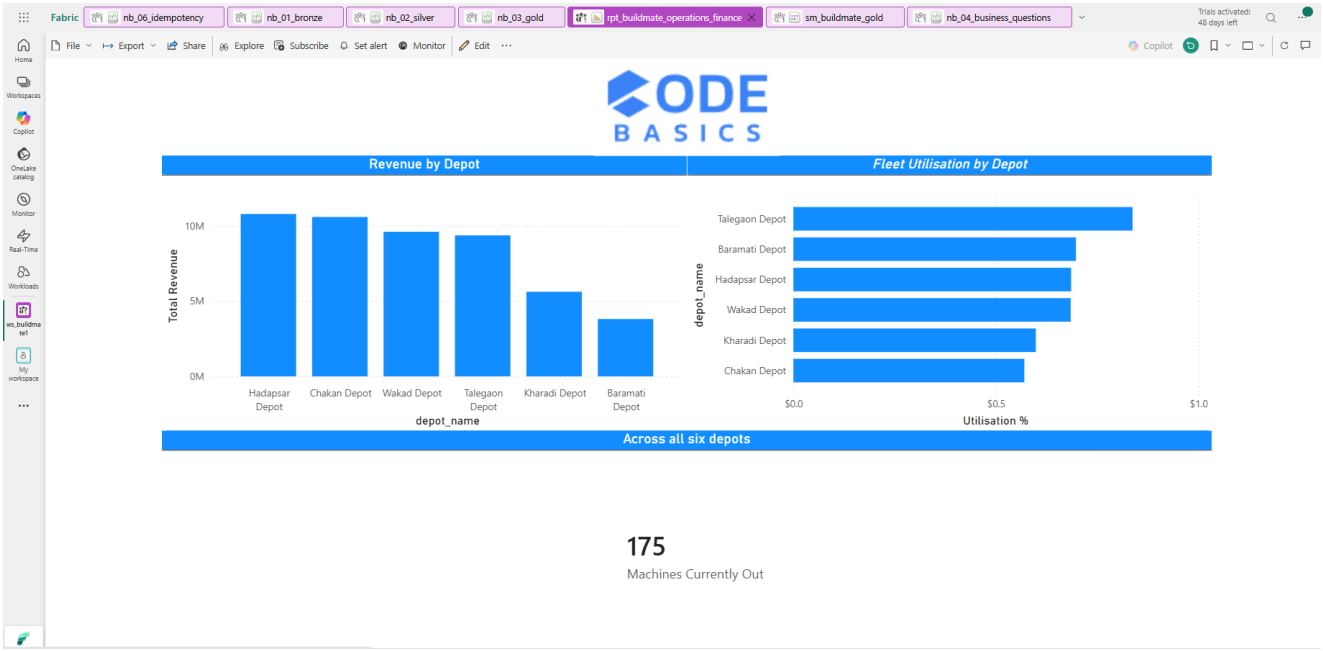
Machines currently out by depot

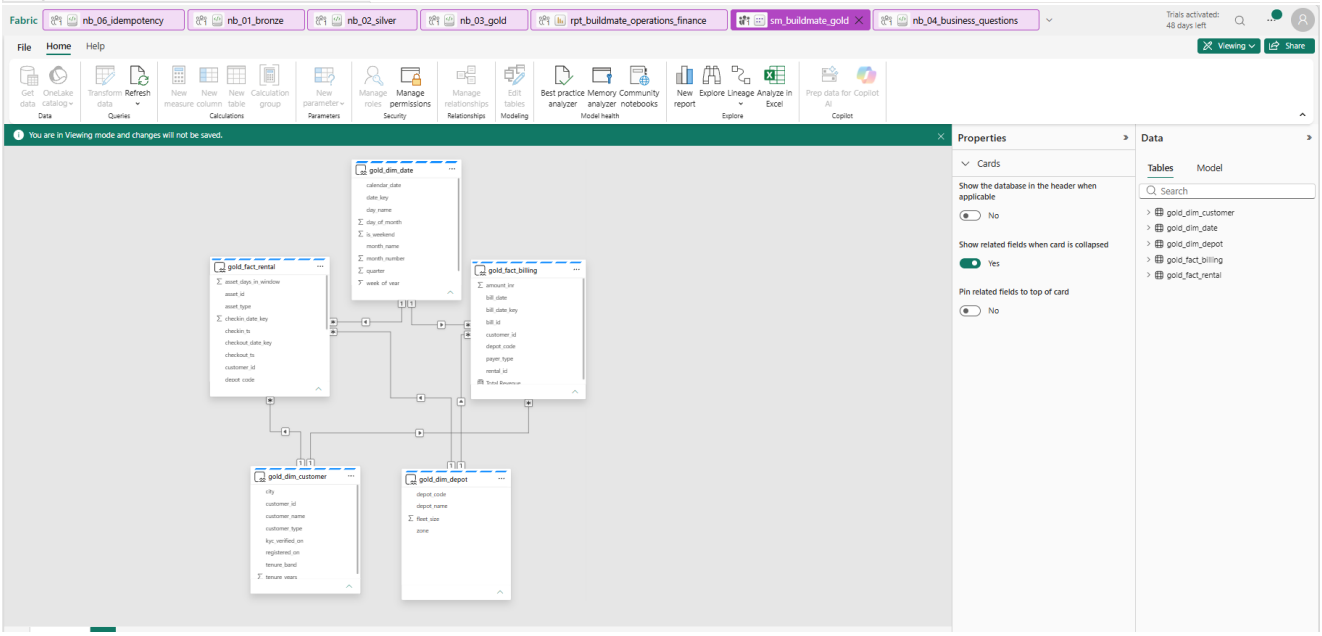
Depot	Currently out
Chakan Depot	45
Hadapsar Depot	40
Wakad Depot	28
Talegaon Depot	27
Baramati Depot	18
Kharadi Depot	17

Stakeholder interpretation for Rhea and Arun

Hadapsar Depot generated the highest total revenue at INR 10,785,755, making it the largest revenue contributor during the period. Talegaon Depot earned the most revenue per rental at approximately INR 90,920, showing a higher-value rental mix despite fewer bills. Across all depots, 175 machines are currently on customer sites, with Chakan holding the highest count at 45. This figure is reliable because the Silver transformation used a null-safe filter that preserved blank check-in timestamps. Had filter(~bad) been used, every active rental would have been removed and the currently-out result would have appeared incorrectly as zero.

Task 5 - Power BI Direct Lake Serving and Shared Capacity





The intended serving design was a Power BI semantic model created directly from the five Gold tables, using Direct Lake so that Power BI reads the same OneLake Delta files without a separate Import copy. The report page was designed to contain revenue by depot, utilisation by depot and a card for the 175 machines currently out.

Shared-capacity discipline

Microsoft Fabric bills workloads through one shared pool of Capacity Units, so Spark notebooks, Data Factory pipelines, SQL queries, semantic models and Power BI reports can compete for the same capacity. A large transformation, repeated pipeline run or long-running Spark session can therefore slow other activities in the workspace. I would monitor overall CU utilisation, workload-specific consumption, utilisation spikes, throttling or queued operations, notebook and pipeline runtimes, and Power BI report responsiveness. Heavy engineering jobs should be optimised or scheduled outside peak reporting periods.

Task 6 - Idempotent Silver Rental Rebuild

A separate notebook, nb_06_idempotency, was used so that the original Silver notebook remained focused on transformation logic. The Silver rental outputs are rebuilt deterministically from the complete Bronze snapshot, deduplicated by rental_id and written with mode('overwrite') rather than append.

- First rebuild: 942 valid rentals and 3 quarantined rentals.
- Second unchanged rebuild: identical counts, proving repeatability.
- No rental ID can appear in both silver_rentals and silver_rentals_quarantine.
- A controlled rentals_2026-07-01.csv test file can be used to prove that only that day's valid new IDs increase Silver.

The combination of rental_id as the deterministic business key, row_number-based deduplication, the same null-safe quality rule, and overwrite writes guarantees idempotency. Re-running the notebook replaces the active Silver snapshot instead of stacking duplicate rows, and quarantined records remain excluded from the valid table.

Final Submission Checklist

- Bronze tables created with all raw source columns stored as strings and lineage preserved.
- Silver counts and all required parse-failure assertions validated.
- Null-safe filtering explicitly demonstrated to preserve 175 active rentals.
- Five Gold star-schema tables created and validated.
- OPTIMIZE, ZORDER and DESCRIBE HISTORY evidence captured.
- Six business queries executed with the required output columns and rounding.
- Revenue and currently-out results reconciled across the model.
- Idempotent rebuild proved through two identical Silver runs.
- Power BI Direct Lake step documented, including the current licensing blocker.

Conclusion

The BuildMate solution demonstrates the complete Fabric engineering lifecycle: one copy of source data in OneLake, faithful Bronze ingestion, careful and null-safe Silver cleaning, a conformed Gold star, Delta optimisation, business reconciliation and rerun-safe processing. The strongest quality result was preserving the 175 live rentals that a careless Boolean filter would have silently deleted.